# GTANAVMESH Edit - User Guide

Version: v0.11.1
Copyright (c) 2026 devsanja

## 1. Purpose

GTANAVMESH Edit is a Blender addon for importing, editing and exporting GTA V YNV navmesh XML files in a CodeWalker-compatible format. It can import XML directly or work with a navmesh imported through Sollumz.

The addon is designed to stay a pure Blender addon. CodeWalker and Sollumz reference behavior was used for compatibility decisions, but the addon does not require CodeWalker at runtime.

> Unofficial community tool. GTANAVMESH Edit is not affiliated with, endorsed
> by, or associated with Rockstar Games or Take-Two Interactive.

### Requirements and external tools

Blender 2.80+ is required. Sollumz is optional and must be installed separately if its importer or property panels are needed. CodeWalker is optional but recommended for inspection and the target workflow. Building the optional ynv_compiler requires the .NET 8 SDK and a compatible local CodeWalker.Core project.

The release ZIP contains only GTANAVMESH Edit; it does not package Sollumz, CodeWalker, CodeWalker.Core, GTA V files or other third-party assets.

Main goals:

- Import CodeWalker .ynv.xml navmesh files directly, and retain compatibility with Sollumz-imported navmeshes.
- Preserve important original metadata such as AreaID, ContentFlags, points, portals, polygon flags, edge flags and edge links.
- Allow face-level flag editing inside Blender.
- Export one selected navmesh or all loaded navmeshes back to .ynv.xml.
- Support geometry-based edge rebuilds for edited navmeshes.
- Create a starter navmesh cell for new work.
- Optionally compile a validated XML export to native RSC7 .ynv with a user-configured local compiler.

## 2. Important Concepts

### Navmesh Cell

A YNV file usually represents one navmesh cell. The file name often looks like navmesh[123][99].ynv.xml.

Each cell has its own:

- AreaID
- ContentFlags
- bounding box (BBMin, BBMax, BBSize)
- polygons
- points
- portals
- edge links to internal polygons and external neighbor cells

## AreaID

AreaID is the numeric cell identifier used inside edge links. It must stay correct when replacing an existing game navmesh.

If the AreaID is wrong, edge links can point to the wrong cell or become invalid.

## Source XML

The Source XML is the original .ynv.xml file that belongs to the imported navmesh.

It is important because it allows the addon to preserve data that is difficult to reconstruct perfectly from Blender geometry alone:

- original edge links
- edge flags
- polygon flags
- portal references
- points
- bounds
- AreaID
- ContentFlags

Face metadata is stored by stable Blender face ID rather than by a transient polygon position. Reordering faces keeps matching metadata aligned; deleting a face removes it from the output instead of restoring it from the source XML.

Manual source selection is the safe default. Select the navmesh, use the Load Source button, and choose the exact original XML. Fix Sources is only an opt-in recovery aid for a moved .blend: enable Auto Find Source and optionally set a narrow Source Search Folder. An unnamed automatic match requires equal polygon count, near-identical bounds and exactly one candidate. It cannot prove that similarly named cells are interchangeable. Always confirm the selected path, AreaID, polygon count and portal count before you edit or export.

## Polygon Flags

Each polygon has CodeWalker-style integer flags. The addon exposes these both as raw flag integers and as bit checkboxes.

Examples of known bits:

- Footpath
- Road
- CellEdge
- FlatGround
- TrainTrack
- ShallowWater
- slope direction bits

Material colors are derived from the Sollumz-style flag interpretation so navmesh surfaces can be visually inspected in Blender.

## Edge Links

Each polygon edge has a link row such as:

4038:5, 4038:5

This points to polygon 5 in area 4038.

An open edge is written as:

16383:16383, 16383:16383

Open edges are allowed on true borders, but too many open edges usually means connectivity was lost.

### Boundary Edges

A boundary edge is an edge used by only one face in the current mesh.

If another loaded navmesh has a matching boundary edge at the same world position, the addon can create a cross-cell edge link.

## 3. Installation

1. In Blender, open Edit > Preferences > Add-ons.
2. Use Install....
3. Select GTANAVMESH-Edit-v0.11.1.zip.
4. Enable GTANAVMESH Edit.
5. Open the 3D View sidebar and use the GTANAVMESH Edit tab.

After updating the addon zip, reload or reinstall the addon so Blender uses the newest code.

## 4. Panel Fields

### AreaID

The navmesh cell ID written into the exported XML.

Use the original Source XML whenever possible so this value stays correct.

### ContentFlags

The XML ContentFlags value is still shown as text, for example:

Polygons, Portals

or:

Polygons, Portals, Unknown8

For direct editing, the panel also provides five toggles:

- Polygons
- Portals
- Vehicle
- Unknown8
- Unknown16

Toggling a known value updates the comma-separated XML value. Other tokens from an imported source remain in the text field, so a new control does not discard an unfamiliar source flag. Use the raw field only when you need to preserve or deliberately author a value outside the named controls.

### Structured Polygon Flags

After selecting a face, use Load Active in Polygon Editor. The editor shows the raw Flags line and the matching individual values:

- Flags1, Flags2, Flags3, Flags4, UnkX and UnkY: byte values (0-255).

- Flags5 (B16): the single supported B16 value (0 or 1).

Numeric fields, the raw Flags line and the bit controls stay synchronized. The visible named bits include Footpath, Road, CellEdge, FlatGround, Water, TrainTrack, ShallowWater, Interior, PathNode, SteepSlope and the eight slope-direction flags. Other controls are labelled as Unknown or Unused where their gameplay meaning is not established; they are still editable raw YNV data, not recommended defaults.

### Structured Edge Flags

Every polygon vertex has one EdgesFlags row. Choose the zero-based Edge index to expose that row as four bounded values:

- Poly1 Unk2 and Poly2 Unk2 (0-3)
- Poly1 Unk3 and Poly2 Unk3 (0-2047)

The values are written back to the selected row in CodeWalker form:

Poly1 Unk2:Poly1 Unk3, Poly2 Unk2:Poly2 Unk3

The raw multi-line EdgesFlags value remains available. Keep exactly one edge link and one edge-flags row for each vertex of the polygon.

### Source XML

Path to the original .ynv.xml file used to preserve metadata.

### Auto Build Edges

For a mesh without prior metadata, the addon can build local same-cell links during export. This is mainly a fallback for newly created simple navmeshes; use an explicit
Rebuild Edges after a topology change.

For existing imported cells, Source XML preservation is preferred.

### Strict Validation

Compatibility validation always rejects unsafe packed-field, bounds, edge and portal data. With Strict Validation disabled, known legacy Rockstar issues such as a zero-area corner or one-way local link are reported as warnings. Enable it to turn those warnings into errors when authoring new geometry.

## 5. Button Reference

### Import YNV XML

Opens an original .ynv.xml and creates the navmesh mesh, point hierarchy and portal hierarchy. The selected XML becomes the Source XML immediately.

You can also reach it from File > Import > YNV NavMesh XML (.ynv.xml).

### New Navmesh

Creates a small starter triangle using the panel’s AreaID and ContentFlags.

Use it only for a genuinely new cell. Set a unique, valid AreaID, create walkable geometry, assign flags and rebuild edges before export. A new mesh does not have authoritative source topology, portal intent or neighbour links.

## Use Selected

Marks the selected mesh as a YNV navmesh and loads its metadata into the panel.

Use this if a mesh should be treated as the active navmesh.

## Apply

Writes the panel metadata to the selected navmesh:

- AreaID
- ContentFlags
- Source XML

## Load Source

Loads an original CodeWalker .ynv.xml as source metadata.

For live faces that exactly match the source, the addon attaches:

- polygon flags
- edge links
- edge flags
- polygon portal references

## Fix Sources

Attempts to find missing Source XML paths for loaded navmeshes.

This is useful when a Blender file was moved or reopened and the addon needs to reconnect a navmesh to its original XML. The addon stores the absolute source path, the path relative to the .blend when possible, and the source file name.

It is deliberately best effort. It does not overwrite a manually chosen source and leaves ambiguous candidates unresolved. Inspect the resulting Source XML, AreaID, polygon count and portal count yourself.

## Validate

Checks the current export model for CodeWalker-compatible structure.

Use this before final export. Toggle Strict Validation when newly authored geometry must also be free of legacy compatibility warnings. It is not a native RSC7 compilation check and it does not prove in-game pathfinding behavior.

## Export XML

Exports the selected navmesh to one .ynv.xml file.

The exporter preserves Source XML data when possible.

## Export All

Exports all loaded navmeshes to a folder.

This is the recommended export when several neighboring cells are loaded. It preserves:

- AreaID
- ContentFlags
- bounds
- points
- portals

- flags
- edge links

It also updates matching cross-cell boundary links without destroying existing internal links.

### Compile YNV

Runs the optional cw_navmesh.compile_ynv operator. It invokes a local ynv_compiler.exe chosen by the user and writes a native .ynv only after the XML has passed the configured validation mode and the resulting bytes have passed an in-memory RSC7 round trip.

Keep the source XML and output binary separate. The addon does not download a compiler, and compilation is not a replacement for Validate or a game test. After compiling, run ynv_compiler --check <output.ynv> to verify the RSC7 file can be read by the CodeWalker.Core-based reader used by the local compiler.

### Build Edges

Builds local edge links inside the same navmesh only.

Use this for simple same-cell geometry, but be careful on imported cells because it does not rebuild external neighbor links.

### Rebuild Edges

Rebuilds edge links from geometry.

It can create:

- internal links between faces in the same mesh
- cross-cell links to matching loaded neighbor cells
- preserved external links to cells that are not loaded

Use this after adding or changing faces, for example when adding a bridge or custom walkway.

### Add Point

Adds a YNV point at the 3D cursor under the current navmesh root.

### Add Portal

Adds a YNV portal at the 3D cursor under the current navmesh root.

## 6. Polygon Editor

The polygon editor works on the active face.

### Load Active

Loads the active face metadata into the editor:

- flags
- edges
- edge flags
- portal refs
- individual Flags1-Flags5, UnkX and UnkY controls
- named polygon bit controls
- the selected EdgesFlags row and its packed components

### Apply

Writes the editor values back to the active face.

For a flag-only change, the synchronized numeric, named-bit and selected-edge controls write to the current active polygon directly. Apply remains the safe explicit step after editing raw Flags, Edges, EdgesFlags or portal rows.

### Copy Flags

Copies the active face flags.

### Paste Flags

Pastes copied flags to all selected faces. If no faces are selected, the flags are pasted to the active face.

## 7. Typical Workflows

### Import and Re-export Without Changes

1. Import the original .ynv.xml.
2. Confirm that the exact original file appears in Source XML.
3. Use Validate.
4. Use Export XML or Export All.

Expected result: bounds, points, flags and edge links should match the original.

### Editing Flags

1. Select a face.
2. Click Load Active.
3. Change named bit checkboxes, Flags1-Flags4, UnkX, UnkY or Flags5 (B16). Use the raw Flags value only for an exact existing value.
4. To edit edge metadata, select an Edge and change the two Unk2 and two Unk3 values for that row.
5. Click polygon Apply after any raw field change.
6. Use Validate.
7. Export.

### Adding a Custom Bridge or Walkway

1. Place or model the bridge/collision surface in Blender.
2. Create navmesh faces on the walkable surface.
3. Join those faces into the correct navmesh mesh, or create a new navmesh cell if needed.
4. Set suitable flags, for example road, footpath or flat ground depending on the use case.
5. Make sure connection edges line up cleanly with the existing navmesh.
6. Run Rebuild Edges.
7. Use Export All if neighbor cells are involved.
8. Validate and compare the exported XML.

Important: if the polygon count changes, the exporter still writes only the current Blender faces. It cannot blindly copy original edge links to new or deleted faces, so Rebuild Edges must create valid connectivity from geometry.

### Compile a Native Replacement

1. Export a validated .ynv.xml to a new output folder.
2. In the addon, choose the local compiler executable and use Compile YNV, or run ynv_compiler <input.ynv.xml> <output.ynv> in PowerShell.
3. Run ynv_compiler --check <output.ynv>.
4. Keep both outputs, inspect the XML in CodeWalker, and test the exact replacement/mod-loading setup in game.

### Collision-to-Navmesh Workflow

Sometimes a collision mesh is placed over an area and selected faces are used as new navmesh geometry.

This can work if:

- the extracted faces are clean and walkable
- the faces are joined into the correct navmesh object or assigned to a correct new cell
- the connection edges are aligned
- flags are set correctly
- edges are rebuilt before export

Avoid tiny gaps, overlapping duplicate faces and disconnected vertices at connection points.

## 8. Export All Safety Rules

Export All is designed to avoid the old problem where cells, bounds or edge data were lost.

Current behavior:

- Each navmesh exports with its own Source XML when available.
- Points and portals are taken only from the matching navmesh root.
- If no clear root exists, Source XML points and portals are used.
- Existing internal edge links are preserved.
- Face metadata is reindexed through stable face IDs; links to deleted faces become open boundaries instead of reviving source geometry.
- Cross-cell boundary links are updated only where geometry matches loaded neighbour cells, with reciprocal links on both loaded cells.

## 9. Troubleshooting

### Bounds are too large

Cause: points or portals from multiple navmeshes may have been included.

Fix: use the updated addon, keep each imported navmesh under its own root, and use Source XML paths.

### Too many 16383:16383 open edges

Cause: edge links were lost or geometry does not connect.

Fix:

- Reload the exact Source XML if the mesh was never bound to it.
- After topology changes, run Rebuild Edges.
- Check for gaps or misaligned boundary edges.

### NPCs do not cross a custom bridge

Possible causes:

- bridge navmesh faces are not connected to the existing navmesh
- flags are wrong
- edge links are open
- the bridge is in the wrong cell
- connection edges do not match neighbor geometry

Use Rebuild Edges, then inspect exported edge links.

### Duplicate AreaID warning

Two loaded navmeshes have the same AreaID.

This is usually wrong unless you intentionally duplicated a cell for testing.

### Polygon count differs from Source XML

The addon preserves matching face metadata but does not blindly reuse a source-only polygon row.

Use Rebuild Edges and validate the result.

## 10. Recommended Checks Before Game Testing

Before putting an exported XML into a game workflow:

1. Run Validate.
2. Use Export All for multi-cell edits.
3. Compare important fields against the original:

- AreaID
- ContentFlags
- BBMin
- BBMax
- point count
- portal count
- edge open/internal/external counts

4. Inspect custom areas for open edges.
5. Keep backups of original XML files.

## 11. Notes

This addon exports CodeWalker-compatible XML and can optionally call a user-configured local compiler for native .ynv output.

For edited geometry, correct navmesh behavior in game still depends on valid YNV compilation and on how the target game setup loads the replacement navmesh.

See YNV_Import_Source_Compile.md for the companion workflow.